



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai-602105

September 2026

CSA0619– Design and Analysis of Algorithms

**Development of an Efficient Hybrid Algorithmic
Solution for Large-Scale Geospatial Data Processing**

Assignment Report

Submitted by

BANDI MOHITH MANOJ (192472018)

YASODHA KRISHNA (192472100)

KODANDA RAM T (192524162)

A. Assignment Information

Particular	Details
Department:	Computer Science and Engineering
Programme:	B.E./B.Tech. Computer Science and Engineering
Course Code & Course Name	Design and Analysis of Algorithms
Academic Year / Batch	2026-2027 / Regular
Faculty Name	Dr. F. Mary Harin Fernandez
Assignment Title	Development of an Efficient Hybrid Algorithmic Solution for Large-Scale Geospatial Data Processing
Date of Issue	31 Aug 2026
Date of Submission	1 September 2026
Maximum Marks	100
Course Outcome(s) – CO	CO1: Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. CO2: Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems CO3: Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication
Bloom's Taxonomy Level	L4 – Analyze, L5 – Evaluate, L6 – Create
SDG Mapping (SDG 1-17)	SDG 9 – Industry, Innovation and Infrastructure
Industry / Societal Relevance	Efficient processing of large-scale real-world data with reduced computational time and resource utilization

B. Assignment Problem / Challenge

Modern infrastructure-planning applications process large volumes of **geospatial coordinate data**, making computationally efficient algorithms essential for identifying spatial relationships among facilities. Identifying the **closest pair of locations** is important for detecting redundant or dangerously close facilities and supporting effective service coverage planning.

Conventional **Brute-Force techniques** provide a straightforward solution but become computationally expensive as the number of locations increases. **Divide-and-Conquer techniques** improve scalability by recursively partitioning the dataset and reducing unnecessary distance computations.

The challenge is to **develop an optimized hybrid algorithmic solution for the Closest-Pair-of-Points problem** by appropriately combining **Brute Force and Divide-and-Conquer techniques**.

The developed solution must determine an effective switching threshold between the two approaches and be supported by **theoretical complexity analysis and experimental evaluation using a real-world geospatial dataset**.

C. Problem Statement

An infrastructure-planning agency maintains a large, publicly available geospatial dataset of facility locations—for example, the OpenFlights airports dataset or a cell-tower/weather-station dataset containing coordinate records. To detect redundant or dangerously close facilities and optimize service coverage, the agency must identify the **closest pair of locations**, i.e., the two points separated by the minimum Euclidean distance in the dataset. Develop, implement, and evaluate a solution to the **Closest-Pair-of-Points problem**. Implement **(i) a Brute-Force algorithm** that evaluates all $n(n-1)/2$ pairs with $O(n^2)$ complexity and **(ii) a Divide-and-Conquer algorithm** that recursively partitions the point set about the median x-coordinate and evaluates points across the dividing strip with $O(n \log n)$ complexity. Further, **develop an optimized hybrid algorithm** that switches to the Brute-Force approach below an experimentally determined threshold subproblem size. Analyze the theoretical efficiency of each approach using **Big-O, Ω , and Θ notations**, and solve the recurrence $T(n) = 2T(n/2) + O(n)$ using the **Master Theorem**. Experimentally evaluate all three algorithms on the selected dataset using increasing input sizes, **for example, 10^3 , 5×10^3 , 10^4 , and higher input sizes wherever computationally feasible**, measuring **execution time, number of distance computations/comparisons, memory utilization, and scalability**. Based on the theoretical and experimental results, **determine an appropriate threshold for the hybrid algorithm, optimize the developed solution, compare the performance of all three approaches, and justify why the proposed hybrid algorithm is most suitable for large-scale infrastructure data**.

Expected Development

The work should **not merely implement the existing Brute-Force and Divide-and-Conquer algorithms**. The solution should demonstrate algorithmic development through:

- **Hybridization of Brute Force and Divide-and-Conquer techniques** for the Closest-Pair-of-Points problem.
- **Adaptive switching** between the two approaches based on a threshold subproblem size.
- **Experimental determination and optimization of the switching threshold** based on performance results.
- **Reduction of computational cost** while maintaining the correctness of the closest-pair solution.
- **Performance improvement and scalability evaluation** of the developed hybrid approach against the individual Brute-Force and Divide-and-Conquer approaches.

D. Requirements and Constraints

The following requirements and constraints shall be considered:

- Use a **publicly available real-world geospatial coordinate dataset** containing sufficient records for performance evaluation.
- Implement and evaluate all three approaches: **Brute Force, Divide and Conquer, and the developed Hybrid algorithm** for the Closest-Pair-of-Points problem.
- Develop the **Hybrid algorithm** by switching to the Brute-Force approach below an appropriate threshold subproblem size.
- Determine and justify the **switching threshold experimentally** based on performance results.
- Perform theoretical efficiency analysis using **Big-O, Ω , and Θ notations** and analyze the

Divide-and-Conquer recurrence using the **Master Theorem**.

- Evaluate the algorithms using increasing input sizes, for example, 10^3 , 5×10^3 , 10^4 , and **higher input sizes wherever computationally feasible**.
- Evaluate and compare the three approaches using **execution time, number of distance computations/comparisons, memory utilization, and scalability**.
- Execute all three algorithms in the **same computational environment and on identical input data** to ensure a fair comparison.
- Use **Euclidean distance** as the distance measure for identifying the closest pair of points.
- Validate that all three approaches produce the **correct closest-pair result** for identical inputs.
- Higher input sizes shall be considered only where computationally feasible, taking into account the $O(n^2)$ **computational cost of the Brute-Force approach**.
- Clearly document the **dataset source, experimental environment, threshold value, assumptions, limitations, and implementation details**.

E. Student Work

1. Problem Understanding and Formulation

1.1 Real-World Geospatial Infrastructure Context

The Closest-Pair-of-Points problem asks for the two distinct locations in a collection whose separation is minimum under a chosen distance metric. In geospatial infrastructure planning, this primitive can be used as an early screening operation for airports, service facilities, weather stations, communication towers, logistics hubs, emergency-response assets, and other geographically distributed facilities. Detecting unusually close facilities can support redundancy analysis, possible spatial-conflict screening, service-coverage planning, and prioritization for further engineering review. The algorithm is therefore a useful example of how a classical algorithmic problem maps to a practical spatial-data processing task.

The central computational challenge is that the most direct method compares every pair of points. For n locations, this requires $n(n-1)/2$ pair evaluations. As the dataset becomes larger, the number of pairwise checks rises quadratically. This makes the problem a suitable case study for comparing the Brute-Force technique with a Divide-and-Conquer technique and then designing a hybrid strategy that combines their strengths.

1.2 Dataset Selection and Source

The selected real-world dataset is the OpenFlights airport database, distributed as airports.dat. OpenFlights describes the airport database as a global collection of airport, station, and ferry-terminal records and documents fields such as Airport ID, Name, City, Country, IATA, ICAO, Latitude, Longitude, Altitude, Timezone, Type, and Source. Latitude and longitude are represented in decimal degrees, making them directly usable as two-dimensional coordinate attributes for the present algorithmic experiment [1].

The downloaded dataset used in this experiment is the OpenFlights airports.dat file. After parsing and validating numeric coordinates, the program retained 7,698 records with valid latitude and longitude values. The largest benchmark size used for the three-algorithm comparison was $n = 7,500$ so that the exhaustive Brute-Force method remained computationally feasible on the test computer.

Dataset Item	Value
Dataset	OpenFlights airports.dat
Source	OpenFlights public airport database [1]
Raw record set	Downloaded airports.dat
Valid coordinate records	7,698

Dataset Item	Value
Largest benchmark size	7,500
Primary fields used	Name, Latitude, Longitude
Coordinate representation	2-D point (x, y)
Distance metric	Euclidean distance

1.3 Expected Input

The algorithm accepts a collection of records that contains valid latitude and longitude values.

Each valid airport record is transformed into a point $p = (x, y)$, where x is latitude and y is longitude for this study. The name of the location is retained with the coordinate so that the final answer can identify the actual pair of airports rather than returning only anonymous points.

1.4 Expected Output

- The names of the two closest locations in the supplied input subset.
- The minimum Euclidean distance between their coordinate pairs.
- The number of distance computations performed by the selected algorithm, where measured.
- A correctness status showing whether the result agrees with the other approaches on identical input data.

1.5 Euclidean Distance Model

For two points $p1 = (x1, y1)$ and $p2 = (x2, y2)$, the Euclidean distance used in the assignment is:

$$d(p1, p2) = \sqrt{[(x1 - x2)^2 + (y1 - y2)^2]}$$

The implementation treats latitude and longitude as planar coordinates because the assignment explicitly specifies Euclidean distance. Therefore, the reported distance is a Euclidean separation in the coordinate representation rather than a geodesic distance over the Earth's curved surface. This distinction is important: a production navigation or aviation application could require a spherical or ellipsoidal distance model, but such a model is outside the scope of this assignment.

1.6 Data Preparation

- Read the OpenFlights airports.dat file using Python CSV parsing.
- Ignore rows with missing or non-numeric latitude or longitude values.
- Retain only coordinates within the valid latitude range $[-90, 90]$ and longitude range $[-180, 180]$.
- Convert each valid record into a point structure containing x , y , and location name.
- Use a reproducible subset-selection method so the Brute-Force, Divide-and-Conquer, and Hybrid implementations receive identical points for each benchmark size.

1.7 Computational Challenges

The Brute-Force algorithm has to evaluate every unordered pair. At $n = 7,500$, the pair count is: $C(7,500, 2) = 7,500 \times 7,499 / 2 = 28,121,250$ distance calculations.

The final experiment records exactly 28,121,250 distance calculations for Brute Force at $n = 7,500$. The large count explains the steep rise in wall-clock time as n increases. Divide and Conquer reduces unnecessary comparisons by recursively solving two smaller subproblems and then examining only a restricted strip around the partition boundary.

1.8 Assumptions, Requirements and Constraints

- The input contains distinct or potentially duplicate coordinate points; the implementation must remain correct for valid records.
- Euclidean distance is the required distance model for all three approaches.
- All algorithms run on identical point subsets and in the same computational environment.
- The Hybrid algorithm uses Brute Force when the recursive subproblem size is less than or

equal to the selected threshold k .

- The largest benchmark is limited to $n = 7,500$ because the Brute-Force baseline becomes increasingly expensive.
- Measured memory is traced peak allocation during the algorithm call, not total operating-system RAM consumption.
- Threshold values are empirical performance parameters for this implementation and environment; they are not universal constants.

2. Application of Course Knowledge

2.1 CO1 – Algorithm Analysis

CO1 is applied through a structured efficiency-analysis process. The first step is identifying a basic operation, here the Euclidean distance evaluation. The analysis then derives the asymptotic cost of the non-recursive Brute-Force method and the recursive Divide-and-Conquer method. Big-O notation is used for an asymptotic upper bound, Ω for a lower bound, and Θ for a tight bound when both sides share the same order. The Divide-and-Conquer recurrence is solved using the Master Theorem.

2.2 Fundamentals of Algorithmic Problem Solving

- Define the input and output precisely before implementation.
- Represent real-world records in a simple point structure suitable for algorithmic processing.
- Establish a straightforward baseline solution before introducing optimization.
- Use correctness checks to ensure that optimization does not change the result.
- Combine theoretical analysis with empirical measurements rather than relying on only one type of evidence.

2.3 CO2 – Brute Force

The Brute-Force solution directly applies the Brute-Force technique by evaluating every possible pair of points. The method is simple, transparent, and useful as a correctness reference because no pair is skipped. Its disadvantage is its quadratic number of pair evaluations, which becomes the dominant cost at large input sizes.

For n points, the number of unordered pairs is $n(n-1)/2$. Therefore, the running time is $\Theta(n^2)$, and the algorithm scales poorly as n increases. The final $n = 7,500$ experiment provides a direct empirical confirmation because 28,121,250 pair computations are recorded.

2.4 CO3 – Divide and Conquer

The Divide-and-Conquer method follows the classical closest-pair strategy. The points are organized by x-coordinate and y-coordinate, the set is divided around the median x-coordinate, each half is solved recursively, and a strip around the dividing line is checked for cross-boundary pairs that could improve the current minimum. The recursion reduces the active problem size while the strip-processing step prevents an exhaustive cross-comparison of the two halves.

2.5 CO1 + CO2 + CO3 Integration in the Hybrid Algorithm

The Hybrid algorithm is the principal development in this case study. CO2 contributes the direct Brute-Force leaf computation, CO3 contributes the recursive partitioning framework, and CO1 provides the complexity-analysis and experimental-method framework used to determine when each method is preferable. The resulting design preserves the asymptotic structure of Divide and Conquer while changing the constant-factor behavior at small recursive subproblems. This is the key distinction between simply implementing three algorithms and developing an integrated algorithmic solution.

3. Solution / Design / Methodology

The implemented workflow follows the assignment sequence exactly: Input Dataset → Data Preparation → Brute-Force Implementation → Divide-and-Conquer Implementation → Hybrid Algorithm Development → Threshold Determination → Complexity Analysis → Experimental Evaluation → Optimization → Performance Comparison → Final Decision.

3.1 Brute-Force Algorithm / Pseudocode

Pseudocode:

BRUTE_FORCE_CLOSEST_PAIR(P)

1. best_distance $\leftarrow \infty$
2. best_pair $\leftarrow \text{null}$
3. for $i \leftarrow 0$ to $n-2$
4. for $j \leftarrow i+1$ to $n-1$
5. $d \leftarrow \text{distance}(P[i], P[j])$
6. if $d < \text{best_distance}$
7. best_distance $\leftarrow d$
8. best_pair $\leftarrow (P[i], P[j])$
9. return (best_pair, best_distance)

The nested loops guarantee that every unordered pair is examined exactly once. This gives a simple and deterministic correctness baseline.

3.2 Divide-and-Conquer Algorithm / Pseudocode

Pseudocode:

DIVIDE_CONQUER_CLOSEST_PAIR(P_x, P_y)

1. if $|P_x| \leq 3$: solve directly by Brute Force
2. split P_x at the median x-coordinate
3. partition P_y into left and right subsets
4. $\delta_L \leftarrow$ recursive solution on left subset
5. $\delta_R \leftarrow$ recursive solution on right subset
6. $\delta \leftarrow \min(\delta_L, \delta_R)$
7. construct strip containing points within δ of the dividing line
8. compare only the relevant neighboring points in the strip
9. return the smallest candidate

The recursive structure divides the original problem into approximately equal halves. The combine step is deliberately restricted to the geometric strip where a cross-boundary improvement is possible.

3.3 Developed Hybrid Algorithm / Pseudocode

Pseudocode:

HYBRID_CLOSEST_PAIR(P_x, P_y, k)

1. if $|P_x| \leq k$: return BRUTE_FORCE_CLOSEST_PAIR(P_x)
2. split P_x about the median x-coordinate
3. form corresponding y-ordered left and right sets
4. left_result $\leftarrow$ HYBRID_CLOSEST_PAIR(left_x, left_y, k)
5. right_result $\leftarrow$ HYBRID_CLOSEST_PAIR(right_x, right_y, k)
6. $\delta \leftarrow \text{minimum}(\text{left_result}, \text{right_result})$
7. build the dividing strip using δ
8. evaluate relevant strip pairs
9. return the best pair and distance

The hybrid switching rule is therefore adaptive with respect to subproblem size. Large subproblems benefit from Divide and Conquer; small subproblems use direct pair enumeration, where the overhead of deeper recursion and repeated small data-structure handling can outweigh the savings from asymptotic partitioning.

3.4 Hybrid Switching Strategy

The switching condition used in the final run is $|P| \leq k$, with the experimentally selected value $k = 8$. This means that whenever a recursive branch reaches eight or fewer points, the algorithm terminates that branch using the Brute-Force solver. Larger branches continue with the Divide-and-Conquer process.

The strategy is motivated by the difference between asymptotic behavior and practical execution cost. Big-O does not capture interpreter overhead, function-call overhead, allocation behavior, cache locality, and the constant factors in sorting and recursive bookkeeping. A small Brute-Force leaf can therefore be faster than continuing recursion even though Brute Force is asymptotically worse.

3.5 Hybrid Algorithm Flowchart

Overall Architecture of the Proposed Adaptive Hybrid Closest-Pair Solution

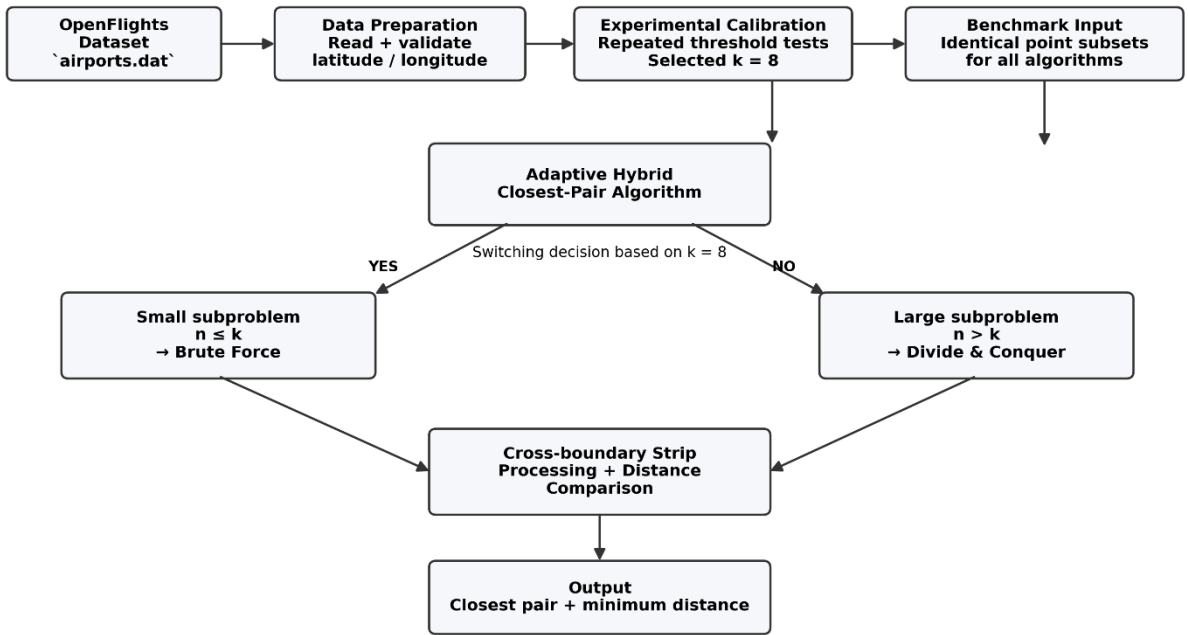


Figure 1. Hybrid Closest-Pair algorithm flowchart showing the threshold-based switch between Brute Force and Divide and Conquer.

3.6 Threshold-Determination Method

Candidate thresholds were evaluated using the same OpenFlights-derived workload and repeated timing measurements. The experiment was designed to identify the threshold that minimized the median execution time across the tested threshold candidates. Using a median reduces the influence of occasional timing spikes caused by normal operating-system or interpreter activity.

The practical selection rule is: $k^* = \arg \min \text{median_runtime}(k)$. The final experiment selected $k = 8$. The threshold should be interpreted as an empirical tuning parameter for this code, dataset, benchmark procedure, and hardware/software environment rather than a mathematically universal constant.

3.7 Mathematical Complexity Analysis

Algorithm	Basic operation / recurrence	Time complexity	Space characteristics
Brute Force	$n(n-1)/2$ pair evaluations	$\Theta(n^2)$	$O(1)$ auxiliary algorithmic space, excluding input storage
Divide and Conquer	$T(n)=2T(n/2)+O(n)$	$\Theta(n \log n)$	Additional structures for ordered views / recursion / strip processing
Hybrid	Same recurrence for large branches; Brute Force for $ P \leq k$	$\Theta(n \log n)$	Similar asymptotic structure to Divide and Conquer

3.8 Master Theorem Derivation

For the Divide-and-Conquer recurrence: $T(n) = 2T(n/2) + O(n)$.

Here $a = 2$, $b = 2$, and $f(n) = O(n)$. Therefore, $n^{\log_b a} = n^{\log_2 2} = n$. Since $f(n)$ has the same asymptotic growth as $n^{\log_b a}$, the equal-growth case of the Master Theorem applies, giving $T(n) = \Theta(n \log n)$.

This theoretical result explains why the Divide-and-Conquer and Hybrid methods remain in a substantially lower runtime range than the quadratic Brute-Force method as the input size increases.

3.9 Algorithmic Design and Optimization Decisions

- Use Brute Force as the correctness baseline because every pair is explicitly checked.
- Use Divide and Conquer for large subproblems to avoid exhaustive pair enumeration.
- Use a configurable threshold rather than a hard-coded textbook leaf size.
- Determine the threshold experimentally with repeated timing measurements.
- Keep input subsets identical across methods so performance differences come from the algorithms rather than different data.
- Separate performance timing from memory tracing so the measurement mechanism does not become part of the primary runtime comparison.
- Limit the largest benchmark to $n = 7,500$ because exhaustive Brute Force would otherwise dominate experiment duration.

4. Use of Modern Tools

4.1 Programming Language and IDE

The solution was implemented in Python and executed locally in Visual Studio Code on a Windows desktop environment. Python was selected because it provides clear support for structured data processing, recursion, precise timing, and result generation while keeping the source code readable for an algorithm-focused academic assignment.

4.2 Data Structures Used

- Point: stores x-coordinate, y-coordinate, and location name.
- Result: stores the current closest pair, distance, and distance-computation count.
- Distance counter: explicitly records every evaluated distance for quantitative comparison.
- X-sorted and Y-sorted point sequences: support the Divide-and-Conquer recursion and strip construction.

4.3 Performance Measurement

Execution time is measured with Python's high-resolution performance-counter mechanism.

Distance computations are counted in the distance-evaluation routine. Peak memory is measured separately using Python memory tracing so that the act of measuring memory does not become part of the primary runtime result. The memory values should therefore be interpreted as traced peak allocation, not as complete process RAM usage.

4.4 Experimental Controls and Reproducibility

- The same cleaned dataset is used for all three methods.
- The same point subset is used for each input size and for each compared algorithm.
- A reproducible selection procedure is used for benchmark subsets.
- Correctness is checked before results are accepted as valid performance observations.
- CSV result files preserve machine-readable measurements for later report generation.
- Threshold tuning is performed before the final hybrid comparison is presented.

4.5 Implementation Evidence

```
OpenFlights data loading and validation
140     return result
141
142
143 def hybrid_closest_pair(
144     points: list[Point], threshold: int = 16, counter: DistanceCounter | None = None
145 ) -> Result:
146     if threshold < 2:
147         raise ValueError("threshold must be >= 2")
148     counter = counter or DistanceCounter()
149     if len(points) < 2:
150         return Result(None, None, math.inf, counter.count)
151     px, py = _prepared(points)
152     result = _closest_recursive(px, py, counter, threshold)
153     result.distance_computations = counter.count
154     return result
155
156
157 def load_openflights_airports(path: str | Path) -> list[Point]:
158     points: list[Point] = []
159     with open(path, "r", encoding="utf-8", newline="") as handle:
160         reader = csv.reader(handle)
161         for row in reader:
162             if len(row) < 8:
163                 continue
164         try:
```

Figure 2. Representative source-code evidence for dataset loading and coordinate validation.

Brute-Force closest-pair implementation

```
52     def distance_sq(self, a: Point, b: Point) -> float:
53         self.count += 1
54         dx = a.x - b.x
55         dy = a.y - b.y
56         return dx * dx + dy * dy
57
58
59 def brute_force(points: list[Point], counter: DistanceCounter | None = None) -> Result:
60     counter = counter or DistanceCounter()
61     if len(points) < 2:
62         return Result(None, None, math.inf, counter.count)
63
64     best_d2 = math.inf
65     best_pair: tuple[Point | None, Point | None] = (None, None)
66     for i in range(len(points) - 1):
67         for j in range(i + 1, len(points)):
68             d2 = counter.distance_sq(points[i], points[j])
69             if d2 < best_d2:
70                 best_d2 = d2
71                 best_pair = (points[i], points[j])
72
73     return Result(best_pair[0], best_pair[1], math.sqrt(best_d2), counter.count)
74
75
76 def _better(a: Result, b: Result) -> Result:
77     return a if a.distance <= b.distance else b
78
```

Figure 3. Representative source-code evidence for the Brute-Force closest-pair implementation.

Divide-and-Conquer recursive implementation

```

87         else:
88             right_y.append(p)
89         return left_y, right_y
90
91
92 def _closest_recursive(
93     px: list[Point],
94     py: list[Point],
95     counter: DistanceCounter,
96     threshold: int | None = None,
97 ) -> Result:
98     n = len(px)
99     if n <= 3 or (threshold is not None and n <= threshold):
100         return brute_force(px, counter)
101
102     mid = n // 2
103     left_x = px[:mid]
104     right_x = px[mid:]
105     mid_x = px[mid].x
106     left_y, right_y = _split_y(py, left_x)
107
108     left_result = _closest_recursive(left_x, left_y, counter, threshold)
109     right_result = _closest_recursive(right_x, right_y, counter, threshold)
110     best = _better(left_result, right_result)
111     delta2 = best.distance * best.distance
112
113     strip = [p for p in py if (p.x - mid_x) ** 2 < delta2]
114     for i in range(len(strip)):
115         j = i + 1
116         while j < len(strip) and (strip[j].y - strip[i].y) ** 2 < delta2:
117             d2 = counter.distance_sq(strip[i], strip[j])
118             if d2 < delta2:
119                 delta2 = d2
120                 best = Result(strip[i], strip[j], math.sqrt(d2), counter.count)
121             j += 1
122
123     best.distance_computations = counter.count
124     return best
125
126
127 def _prepared(points: list[Point]) -> tuple[list[Point], list[Point]]:
128     px = sorted(points, key=lambda p: (p.x, p.y, p.name))
129     py = sorted(points, key=lambda p: (p.y, p.x, p.name))
130     return px, py
131
132
133 def divide_and_conquer(points: list[Point], counter: DistanceCounter | None = None) -> Result:
134     counter = counter or DistanceCounter()
135     if len(points) < 2:
136         return Result(None, None, math.inf, counter.count)
137     px, py = _prepared(points)
138     result = _closest_recursive(px, py, counter)

```

Figure 4. Representative source-code evidence for the Divide-and-Conquer closest-pair implementation.

Robust threshold search and experimental setup

```

255         "distance_computations": counts[0],
256         "distance": result.distance,
257         "pair_1": result.p1.name if result.p1 else "",
258         "pair_2": result.p2.name if result.p2 else "",
259     }
260
261
262 def threshold_search(
263     points_by_size: dict[int, list[Point]],
264     thresholds: Iterable[int],
265     repeats: int,
266 ) -> tuple[list[dict[str, object]], int]:
267     """Select a robust threshold using median runtime over multiple input sizes."""
268     rows: list[dict[str, object]] = []
269     for threshold in thresholds:
270         per_size_medians: list[float] = []
271         total_comparisons = 0
272         for size in sorted(points_by_size):
273             points = points_by_size[size]
274             times: list[float] = []
275             counts: list[int] = []
276             for _ in range(repeats):
277                 _, elapsed, count = _timed_call(points, "Hybrid", threshold)
278                 times.append(elapsed)
279                 counts.append(count)
280             per_size_medians.append(statistics.median(times))
281             total_comparisons += statistics.median(counts)
282         rows.append(
283             {
284                 "threshold": threshold,
285                 "sizes_tested": ";".join(str(s) for s in sorted(points_by_size)),
286                 "median_time_seconds_across_sizes": statistics.median(per_size_medians),
287                 "mean_time_seconds_across_sizes": statistics.mean(per_size_medians),
288                 "median_distance_computations": total_comparisons / len(points_by_size),
289             }
290         )
291
292     best = min(
293         rows,
294         key=lambda row: (
295             float(row["median_time_seconds_across_sizes"]),
296             int(row["threshold"]),
297         ),
298     )
299     return rows, int(best["threshold"])
300
301
302 def save_csv(path: Path, rows: list[dict[str, object]]) -> None:
303     path.parent.mkdir(parents=True, exist_ok=True)
304     if not rows:
305         return
306     with path.open("w", encoding="utf-8", newline="") as handle:
307         writer = csv.DictWriter(handle, fieldnames=list(rows[0].keys()))
308         writer.writeheader()
309         writer.writerows(rows)
310
311
312 def make_subsets(all_points: list[Point], sizes: list[int], seed: int) -> dict[int, list[Point]]:
313     rng = Random(seed)
314     shuffled = list(all_points)
315     rng.shuffle(shuffled)
316     return {size: shuffled[:size] for size in sorted(set(sizes)) if size <= len(shuffled)}
317
318
319 def write_environment(results_dir: Path, dataset_label: str, point_count: int) -> None:

```

Figure 5. Representative source-code evidence for threshold tuning and benchmark configuration.

The implementation evidence demonstrates the required use of modern computational tools and provides a traceable link between the algorithm descriptions and the executable program.

5. Results and Validation

5.1 Experimental Configuration

Parameter	Final experiment
Dataset	OpenFlights airports.dat
Valid coordinate records	7,698
Largest benchmark size	7,500
Selected hybrid threshold	$k = 8$
Distance measure	Euclidean
Algorithms compared	Brute Force, Divide and Conquer, Hybrid
Primary timing statistic	Median execution time
Correctness criterion	Same closest pair and minimum distance

5.2 Increasing Input Sizes

The experiment evaluates increasing input sizes from small subsets through $n = 7,500$. The purpose is not only to identify the fastest method at one input size but to observe the scaling trend. The execution-time plots show a steep rise for Brute Force, while Divide and Conquer and Hybrid remain in a much lower range. This pattern is consistent with the theoretical difference between $\Theta(n^2)$ and $\Theta(n \log n)$.

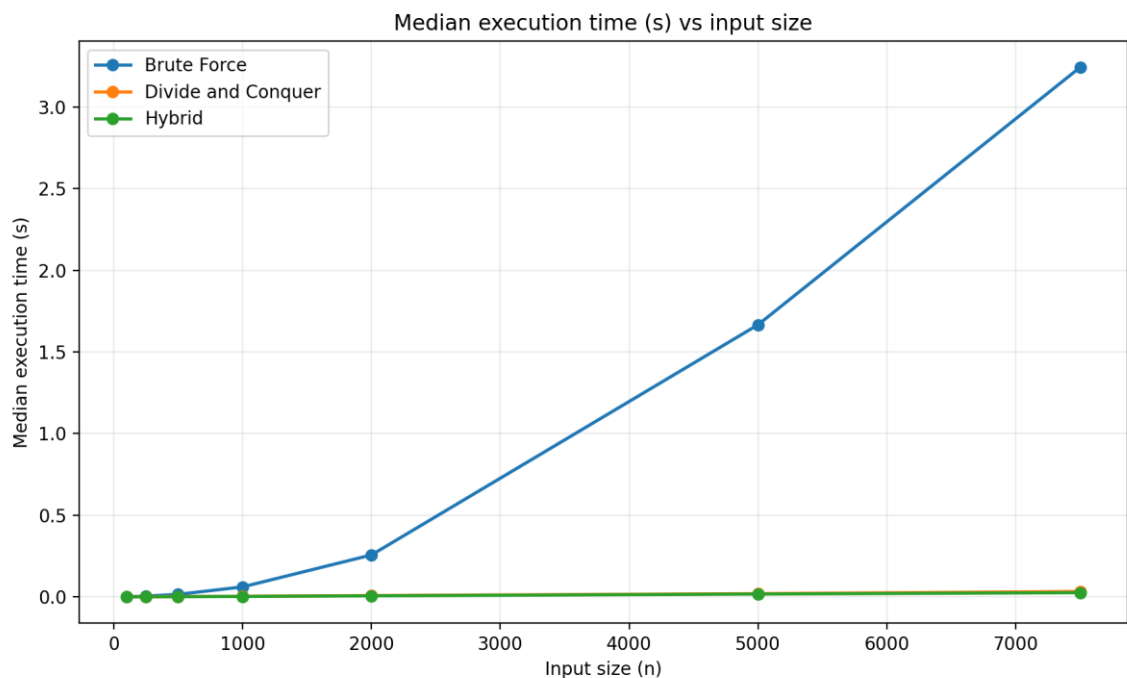


Figure 6. Median execution time versus input size for Brute Force, Divide and Conquer, and Hybrid.

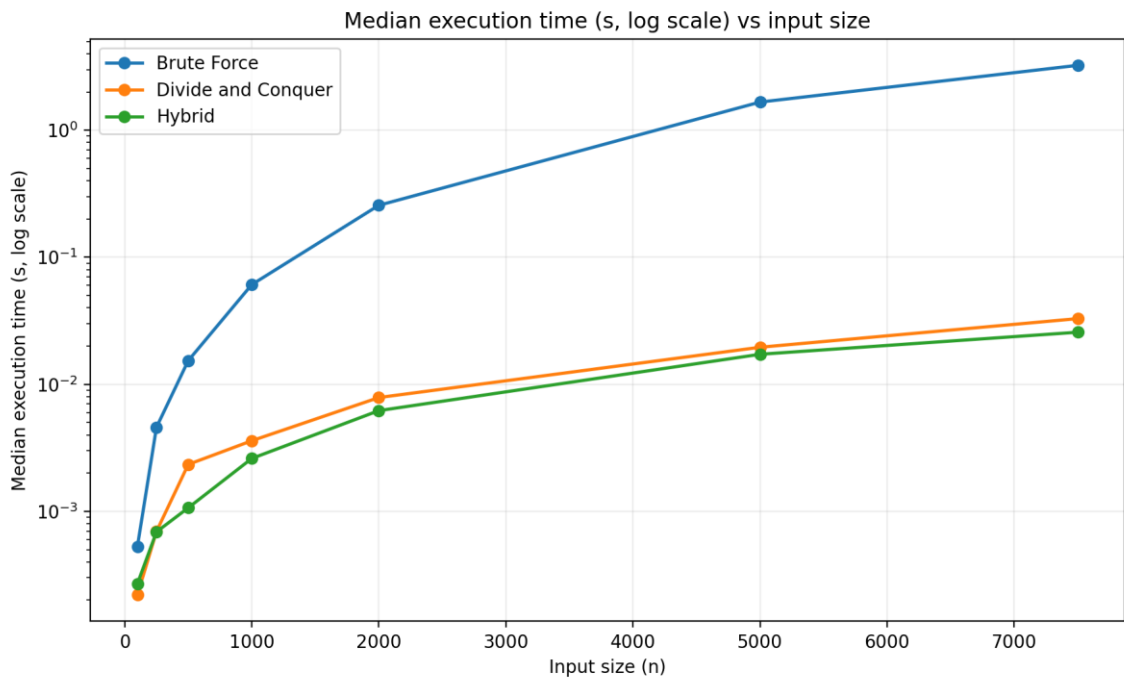


Figure 7. Median execution time versus input size on a logarithmic vertical scale.

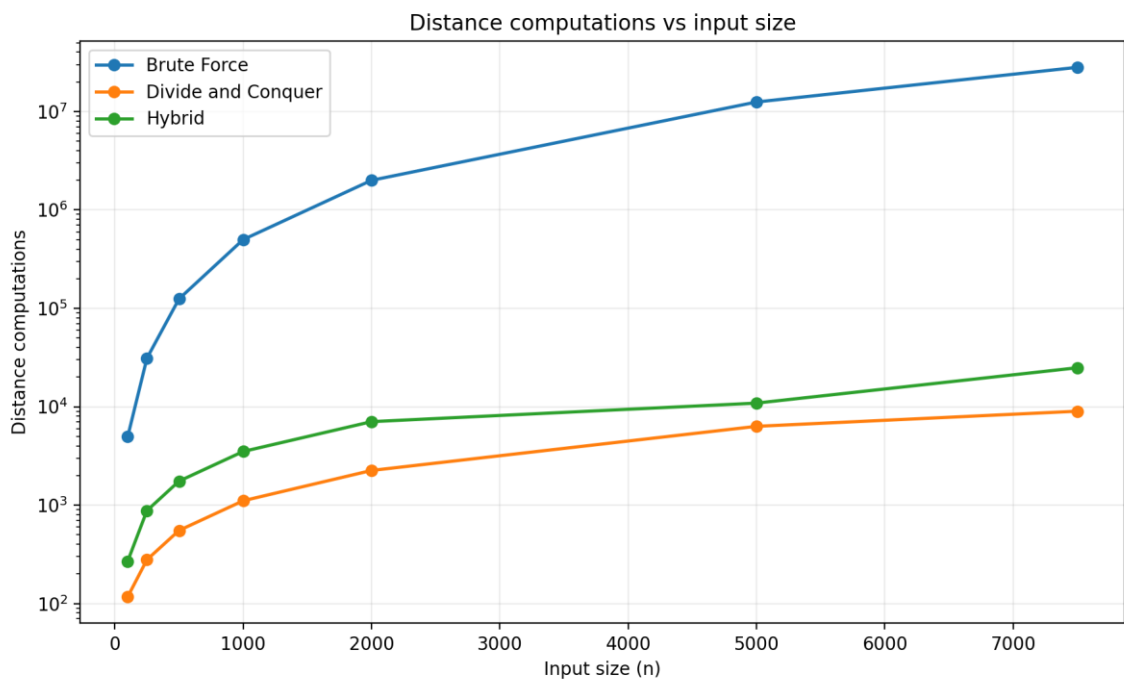


Figure 8. Distance computations versus input size, showing the strong quadratic growth of Brute Force.

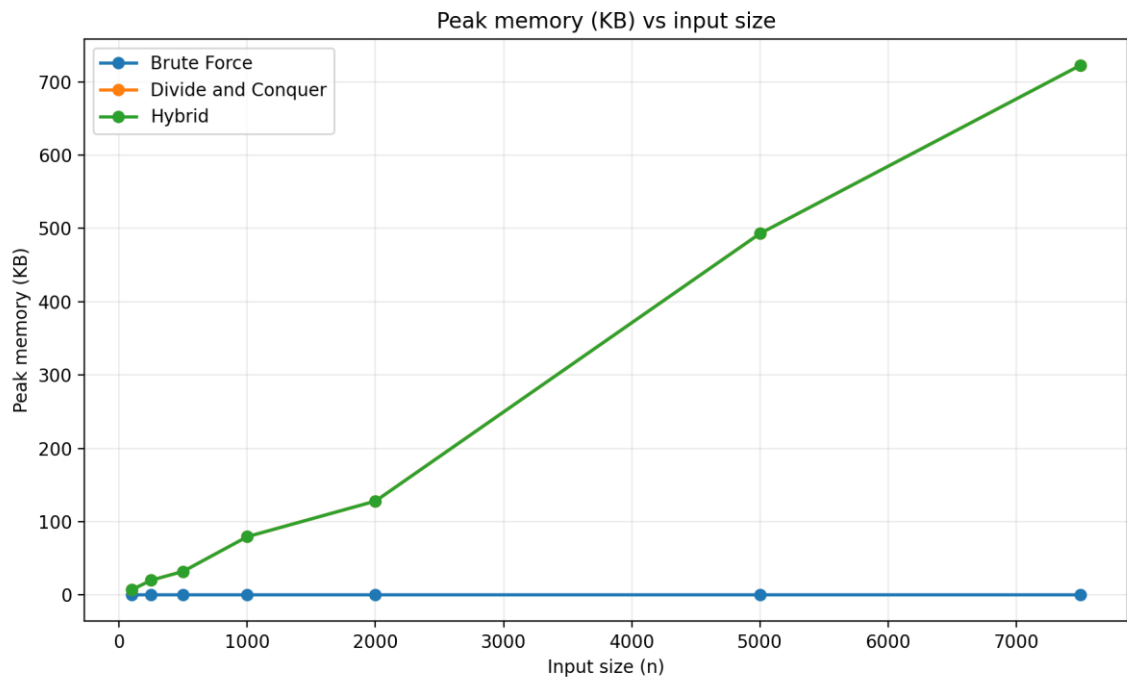


Figure 9. Traced peak memory versus input size. These values represent measured allocations during the algorithm call, not total system RAM.

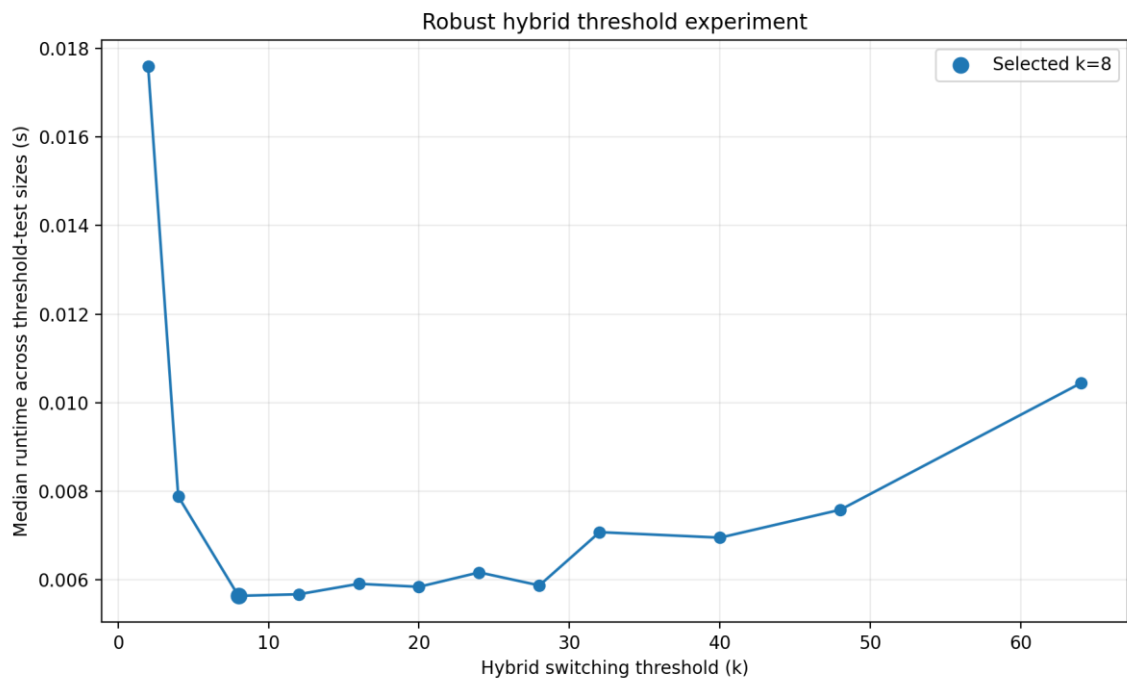


Figure 10. Robust hybrid threshold experiment. The minimum measured threshold-test runtime occurs at $k = 8$.

5.3 Final Performance at $n = 7,500$

Algorithm	Median time (s)	Distance calculations	Peak memory (KB)	Closest pair	Distance
Brute Force	3.242633	28,121,250	0.2	Shute Harbour Airport – Whitsunday Island Airport	0.001242359449
Divide and conquer	0.032881	9,002	722.5	Shute Harbour Airport – Whitsunday Island Airport	0.001242359449
Hybrid ($k = 8$)	0.025702	24,887	722.5	Shute Harbour Airport – Whitsunday Island Airport	0.001242359449

5.4 Derived Performance Indicators

At $n = 7,500$, Hybrid is 21.83% faster than Divide and Conquer based on median execution time, calculated as $(0.032881 - 0.025702) / 0.032881 \times 100$. Relative to Brute Force, Hybrid reduces median time by approximately 99.21%. Although Hybrid performs more measured distance computations than pure Divide and Conquer, it still records the lower elapsed time. This demonstrates that total runtime depends on more than the count of one basic operation; recursion overhead, data movement, and subproblem handling also contribute.

The Brute-Force distance-count reduction achieved by Hybrid at $n = 7,500$ is approximately 99.91%: $(28,121,250 - 24,887) / 28,121,250 \times 100$. Divide and Conquer reduces the count even further, to 9,002 distance calculations. The engineering decision therefore uses multiple metrics instead of assuming that the smallest operation count must always correspond to the shortest wall-clock time.

5.5 Correctness Validation

For each benchmark input size, all three algorithms were run on the identical point subset and their returned closest pair and minimum distance were compared. The final benchmark reports consistent results. At $n = 7,500$, all three methods identify Shute Harbour Airport and Whitsunday Island Airport and return the same coordinate-space Euclidean distance of 0.001242359449.

This agreement validates the hybridization step: changing the way the recursion terminates does not change the final mathematical result. Correctness is therefore maintained while the practical execution time is improved.

5.6 Memory Interpretation

The memory graph must be interpreted carefully. The reported values are traced peak allocations during the algorithm call. Brute Force stays near zero additional traced allocation because it avoids the same auxiliary sorted views and recursive structures. Divide and Conquer and Hybrid use additional memory associated with recursive processing, ordered point views, and strip-related structures. The equality of the reported 722.5 KB values for Divide and Conquer and Hybrid at $n = 7,500$ is therefore consistent with the fact that Hybrid retains the same large-input recursive framework and only changes the leaf strategy.

6. Analysis and Engineering Decision

6.1 Comparison of the Three Algorithms

Aspect	Brute Force	Divide and Conquer	Hybrid
Design	Exhaustive pair enumeration	Recursive partition + strip processing	Divide and Conquer with Brute-Force leaves
Asymptotic time	$\Theta(n^2)$	$\Theta(n \log n)$	$\Theta(n \log n)$
$n = 7,500$ median time	3.242633 s	0.032881 s	0.025702 s
$n = 7,500$ distance calculations	28,121,250	9,002	24,887
$n = 7,500$ peak traced memory	0.2 KB	722.5 KB	722.5 KB
Main strength	Simple and transparent	Excellent scalability	Best measured time in final experiment
Main limitation	Poor scalability	Higher implementation complexity	Requires empirical threshold tuning

6.2 Relationship Between Theory and Experiment

The experimental results follow the expected theoretical trend. Brute Force grows much more rapidly as n increases because its work is proportional to the number of point pairs. Divide and Conquer remains far below the quadratic method because its recurrence is $\Theta(n \log n)$. Hybrid also follows the same large-input asymptotic order, but the experimentally selected threshold changes the practical constant factor.

Real measurements are not expected to match ideal mathematical curves exactly. Python interpreter overhead, object allocation, memory behavior, cache locality, operating-system scheduling, and branch behavior introduce noise and constant-factor effects. The important observation is the persistent separation between the quadratic baseline and the two $n \log n$ methods.

6.3 Effect of Increasing Input Size

At small input sizes, the absolute runtime differences are small enough that implementation overhead can influence which method is fastest. As input size increases, the quadratic growth of Brute Force becomes dominant. The $n = 7,500$ result illustrates the magnitude of this effect: Brute Force requires 3.242633 seconds compared with 0.032881 seconds for Divide and Conquer and 0.025702 seconds for Hybrid.

6.4 Impact of the Switching Threshold

The threshold experiment demonstrates a U-shaped practical trade-off. If k is too small, the algorithm keeps recursing into tiny subproblems and pays additional recursion and data-management overhead. If k is too large, the algorithm replaces too much of the scalable recursion with direct pair enumeration, increasing local quadratic work. The experiment therefore searches for a point where the two costs balance. The measured optimum is $k = 8$ for this implementation and dataset.

6.5 Advantages and Limitations

- Brute Force: easiest to implement and verify, but unsuitable for large-scale workloads because of $\Theta(n^2)$ growth.
- Divide and Conquer: strong asymptotic scalability and very low distance count, but involves more complicated data handling and recursive bookkeeping.
- Hybrid: preserves $\Theta(n \log n)$ asymptotic behavior while improving the measured wall-clock time in the final experiment; however, its threshold must be calibrated and can depend on the execution environment.

6.6 Final Engineering Decision

For the studied OpenFlights workload, the Adaptive Hybrid Closest-Pair algorithm with $k = 8$ is the selected engineering solution. At $n = 7,500$, it records a median execution time of 0.025702 s, compared with 0.032881 s for Divide and Conquer and 3.242633 s for Brute Force. Thus Hybrid is approximately 21.83% faster than the pure Divide-and-Conquer implementation and approximately 99.21% faster than Brute Force on the final benchmark.

The selection is based on the combined evidence of correctness, execution time, distance computations, memory behavior, scalability, and threshold experimentation. Pure Divide and Conquer has the lowest distance count, but Hybrid has the lowest measured elapsed time. For the stated engineering objective—efficient large-scale geospatial processing—the lower wall-clock time of Hybrid is the decisive practical advantage, while the retained $\Theta(n \log n)$ large-input structure prevents the quadratic collapse of Brute Force.

7. Broader Considerations

7.1 Efficient Utilization of Computational Resources

The hybrid design avoids exhaustive pair enumeration across the full dataset and uses direct Brute Force only where the subproblem is small. This combination is intended to reduce CPU time for repeated spatial queries while keeping the algorithm simple enough to inspect and validate.

7.2 Reduction in Unnecessary Distance Computations

At $n = 7,500$, Brute Force performs 28,121,250 distance calculations, while Hybrid performs 24,887. The measured reduction relative to Brute Force is approximately 99.91%. Divide and Conquer performs 9,002 distance calculations, showing that operation count and elapsed time are related but not identical performance objectives.

7.3 Scalability for Large Geospatial Datasets

The main scalability contribution is replacing the quadratic pair-enumeration structure with a $\Theta(n \log n)$ recursive framework for large subproblems. This matters as a system grows from thousands of locations toward tens of thousands or more. The exact timing benefit depends on implementation and hardware, but the asymptotic structure provides a much stronger foundation for scale than exhaustive comparison.

7.4 Infrastructure Planning and Practical Applicability

The closest-pair primitive can act as an early spatial screening step. For example, an infrastructure authority could use it to locate two facilities that are unusually close, identify candidate redundant assets, highlight potential spatial-conflict cases, or generate inputs for

more detailed domain-specific analyses. The same algorithmic framework is reusable for weather stations, public facilities, communication sites, logistics hubs, or other coordinate datasets.

7.5 Sustainable Computational Development

Reducing unnecessary computations supports more efficient use of processor time and makes repeated analysis more practical. In large-scale data systems, algorithmic efficiency can reduce the computational work required per query, which contributes to resource-aware and sustainable software design.

7.6 SDG 9 – Industry, Innovation and Infrastructure

The work relates directly to SDG 9, Industry, Innovation and Infrastructure, through the use of algorithmic innovation to improve infrastructure-data processing. The proposed hybrid solution demonstrates how a classical computational problem can be adapted into a practical, data-driven engineering workflow in which scalability and resource efficiency are explicit design objectives. Such techniques can support intelligent infrastructure planning by making repeated spatial analysis faster and more computationally manageable.

8. Conclusion

This case study addressed the Closest-Pair-of-Points problem using a real-world OpenFlights airport dataset. The study implemented and compared a Brute-Force algorithm, a Divide-and-Conquer algorithm, and a developed Adaptive Hybrid algorithm. The hybrid design integrates Brute Force for small recursive subproblems with Divide and Conquer for larger subproblems and determines the switching threshold experimentally.

The theoretical analysis established $\Theta(n^2)$ time for Brute Force and $\Theta(n \log n)$ time for Divide and Conquer. The Hybrid algorithm retains $\Theta(n \log n)$ asymptotic behavior while changing practical constant factors through threshold-based leaf handling. In the final experiment, the selected threshold was $k = 8$.

At $n = 7,500$, all three algorithms returned the same closest pair—Shute Harbour Airport and Whitsunday Island Airport—with Euclidean distance 0.001242359449. Brute Force required 3.242633 s and 28,121,250 distance calculations; Divide and Conquer required 0.032881 s and 9,002 calculations; Hybrid required 0.025702 s and 24,887 calculations. Hybrid was therefore the fastest method in the final experiment, while Divide and Conquer produced the smallest distance count.

The principal limitation is that the exact threshold is implementation- and environment-dependent. Another limitation is the planar Euclidean treatment of latitude and longitude. Future optimization could investigate geodesic distance, improved memory management, parallel processing, spatial indexing structures, vectorization, or adaptive threshold learning across changing workloads.

9. Student Reflection

9.1 Student 1 – BANDI MOHITH MANOJ (192472018)

1. Learning beyond classroom algorithms: I learned that implementing an algorithm from a textbook and engineering a useful solution are different tasks. The most important additional learning was understanding how a hybrid algorithm can combine two known techniques to obtain better practical behavior. I also learned that asymptotic complexity alone does not determine actual runtime; threshold tuning, recursion overhead, memory behavior, and measurement methodology matter.
2. Challenges and how they were addressed: The main challenge was finding a threshold that improved practical performance without making the algorithm overly dependent on Brute

Force. I addressed this by testing several candidate thresholds using repeated measurements and comparing their median runtime rather than selecting a value arbitrarily.

3. Future improvement: With additional resources, I would test larger datasets, investigate parallel execution, and compare a learned or workload-adaptive threshold against the fixed $k = 8$ setting. I would also examine geodesic distance to make the spatial model more realistic for global geographic data.
4. SDG 9 contribution: The project demonstrates how algorithm optimization can make infrastructure data processing faster and more scalable. By reducing computational work and improving response time for spatial analysis, the approach supports intelligent and efficient infrastructure-management workflows.

9.2 Student 2 – KODANDA RAM T (192524162)

1. Learning beyond classroom algorithms: I learned the importance of data preparation and experimental controls. A correct algorithm can still produce weak experimental evidence if the inputs are inconsistent or if the benchmark procedure changes between methods. Using identical subsets and reproducible selection made the comparison more meaningful.
2. Challenges and how they were addressed: Preparing the real-world dataset required handling missing and invalid coordinates. I addressed this by validating latitude and longitude ranges and retaining only usable records. Another challenge was measuring performance fairly, so the benchmark process was kept consistent across algorithms.
3. Future improvement: With more computing resources, I would expand the benchmark beyond 7,500 points, compare additional real-world datasets, and study the effect of point density and geographic clustering on the optimal threshold.
4. SDG 9 contribution: Reliable processing of infrastructure-location data can help organizations analyze facilities faster. A scalable closest-pair routine can serve as one component within larger intelligent planning and monitoring systems.

9.3 Student 3 – M YASODHA KRISHNA (192472100)

1. Learning beyond classroom algorithms: I learned how theoretical complexity, empirical testing, and engineering judgment must be connected. The most interesting result was that Divide and Conquer used fewer distance calculations than Hybrid, yet Hybrid had the lower wall-clock time. This showed me that a single metric is not enough to judge algorithm quality.
2. Challenges and how they were addressed: A major challenge was interpreting the performance graphs without overclaiming. We addressed this by using the measured values, distinguishing between distance-count metrics and elapsed time, and treating $k = 8$ as an empirical optimum for the tested environment rather than a universal constant.
3. Future improvement: I would evaluate more advanced spatial indexing, investigate parallel implementations, and perform repeated experiments under controlled CPU-load conditions to reduce timing variability.
4. SDG 9 contribution: The project links algorithmic innovation with infrastructure efficiency. Better spatial processing can support faster analysis of facilities and locations, contributing to intelligent infrastructure planning and more computationally efficient systems.

10. References

- [1] OpenFlights, “Airport, airline and route data,” OpenFlights. Available: <https://openflights.org/data>. Accessed September 2026.
- [2] M. I. Shamos and D. Hoey, “Closest-point problems,” in Proceedings of the 16th Annual IEEE Symposium on Foundations of Computer Science, 1975, pp. 151–162.
- [3] J. Kleinberg and E. Tardos, Algorithm Design, Pearson, 2006.
- [4] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed., MIT Press, 2022.
- [5] Python Software Foundation, “Python Documentation,” <https://docs.python.org/>. Accessed September 2026.
- [6] Visual Studio Code Documentation, Microsoft, <https://code.visualstudio.com/docs>. Accessed September 2026.
- [7] AI-assisted tool disclosure: OpenAI ChatGPT was used to assist with report organization, wording refinement, code/report review, and presentation of the algorithmic analysis. Experimental execution, dataset processing, benchmark measurements, and result validation were performed in the local Python environment used for this submission.

F. Assessment Rubric

Criteria	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Problem Understanding & Algorithmic Formulation	10	Clearly identifies the problem, requirements, inputs, outputs, constraints and computational challenges and formulates them appropriately for solution development.	Problem and major requirements are correctly formulated with minor gaps.	Basic formulation is provided, but requirements or constraints are partially identified.	Problem is poorly understood or inadequately formulated.
Algorithmic Solution Design	15	Designs a clear, logical and feasible solution strategy with appropriate algorithmic techniques, pseudocode/flowchart and well-justified design decisions.	Appropriate solution design with minor gaps in logic, representation or justification.	Basic solution design is presented but lacks completeness or clear justification.	Solution design is inappropriate, incomplete or unclear.
Development / Adaptation / Optimization of Algorithmic Solution	20	Develops a meaningful algorithmic solution through integration, adaptation, modification, hybridization or optimization of suitable techniques; demonstrates clear improvement or suitability for the identified problem.	Develops an appropriate solution with meaningful adaptation or improvement, with minor limitations.	Solution demonstrates limited adaptation or development beyond standard implementation.	Merely reproduces an existing algorithm with little or no meaningful development or adaptation.
Implementation & Modern Tool Usage	15	Developed solution is correctly and efficiently implemented using appropriate programming/computational tools and handles relevant input conditions reliably.	Implementation is functional with minor logical or efficiency issues.	Core implementation works but contains limitations or incomplete functionality.	Implementation is incomplete, unreliable or non-functional.
Efficiency, Testing & Validation	15	Performs appropriate complexity evaluation and systematic testing using suitable data/input sizes; validates correctness, efficiency and scalability with clear evidence.	Complexity evaluation and testing are mostly appropriate with minor gaps.	Basic testing and efficiency evaluation are provided but lack sufficient validation.	Testing, complexity evaluation or validation is inadequate or substantially incorrect.
Performance Improvement	15	Demonstrates the effectiveness of the	Good performance	Basic comparison is	Improvement is not

, Comparison & Final Decision		developed solution through appropriate comparison; critically evaluates performance, limitations and trade-offs and justifies the final algorithmic decision with evidence.	comparison and justification with minor limitations.	provided, but improvement or final justification lacks depth.	demonstrated or the final algorithmic decision is unsupported.
Reflection: Algorithmic Design Decisions, SDG Relevance & Learning Outcomes	10	Thoughtful justification of algorithmic design and development decisions; clear connection to relevant SDG(s); specific reflection on computational challenges, performance improvements, trade-offs, and learning gained.	Appropriate justification of algorithmic decisions; SDG relevance, challenges, improvements, and learning are discussed but lack depth.	Reflection is present but generic; limited justification of algorithmic decisions, improvements, SDG relevance, or learning outcomes.	No genuine reflection is provided, or the reflection lacks meaningful connection to algorithm development, SDG relevance, challenges, or learning outcomes.
Total	100				

G. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem Understanding & Algorithmic Formulation	CO1	PO1, PO2	L4	10
Algorithmic Solution Design	CO2, CO3	PO2, PO3	L4, L6	15
Development / Adaptation / Optimization of Algorithmic Solution	CO2, CO3	PO2, PO3	L5, L6	20
Implementation & Modern Tool Usage	CO2, CO3	PO3, PO5	L3, L6	15
Efficiency, Testing & Validation	CO1, CO2, CO3	PO2, PO4	L4, L5	15
Performance Improvement, Comparison & Final Decision	CO1, CO2, CO3	PO2, PO4	L4, L5	15
Reflection: Algorithmic Design Decisions, SDG Relevance & Learning Outcomes	CO1, CO2, CO3	PO7, PO10, PO12	L4, L5	10

H. Faculty Evaluation & Continuous Improvement

CO Attainment

Performance Level	Criterion
Level 3	$\geq 70\%$
Level 2	60–69%
Level 1	50–59%
Level 0	$< 50\%$

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well:

Areas requiring improvement:

Corrective / Improvement Action: